

CEMU: Enabling Full-System Emulation of Computational Storage Beyond Hardware Limits

Qiuyang Zhang*
Huazhong University of Science and Technology
Wuhan, China

Jiapin Wang*
Huazhong University of Science and Technology
Wuhan, China

You Zhou†
Huazhong University of Science and Technology
Wuhan, China

Peng Xu
Research Center for High Efficiency Computing Infrastructure
Zhejiang Lab
Hangzhou, China

Kai Lu†
Huazhong University of Science and Technology
Wuhan, China

Jiguang Wan
Huazhong University of Science and Technology
Wuhan, China

Fei Wu
Huazhong University of Science and Technology
Wuhan, China

Tao Lu†
DapuStor
Shenzhen, China

Abstract

Computational storage drives (CSDs) present a promising approach to improve system performance through near data processing in SSDs. However, current research platforms are fragmented and inadequate to explore the full design space of CSD systems. Existing hardware and emulator platforms are constrained by physical compute resources, while simulators lack full-system fidelity. To address the problems, we introduce CEMU, a new software-based CSD emulation platform that enables full-system research. It consists of a CSD device emulator and a CSD-oriented software stack. Through a novel virtual machine freezing mechanism, CSD emulation achieves high configurability. While the CSD can utilize the host CPU to physically perform computation to preserve full-system behaviors, the computational delay can be modeled separately to emulate CSDs with CPU-unbounded high computing power. The software stack is designed with two principles, adhering to recent industry CSD standards and being compatible with the existing I/O stack, which is achieved via a newly developed file system FDMFS. We verify CEMU's emulation fidelity across a range of applications by benchmarking against actual CSD hardware, demonstrating average end-to-end performance accuracy of 95% or higher. We also use two case studies on large language model training and LevelDB to demonstrate that CEMU is effective in exploring CSD system research and can uncover insights that have not been discovered in previous research platforms.

CCS Concepts: • **Information systems → Storage architectures**; • **Computing methodologies → Modeling and simulation**; • **Hardware → Analysis and design of emerging devices and systems**.

Keywords: Computational Storage, Emulation Platform, Software Stack, Storage Systems

ACM Reference Format:
Qiuyang Zhang, Jiapin Wang, You Zhou, Peng Xu, Kai Lu, Jiguang Wan, Fei Wu, and Tao Lu. 2026. CEMU: Enabling Full-System Emulation of Computational Storage Beyond Hardware Limits. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3779212.3790137

*Both authors contributed equally to this research.
†Corresponding authors: You Zhou, Kai Lu, and Tao Lu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790137

1 Introduction

The exponential growth of data and the surge in AI have led to an increasing demand for computational power and energy efficiency. Traditional computing models require loading large amounts of data from storage devices to the host for processing and then writing the results back. Data movements introduce significant performance overhead and energy consumption, which could cause system bottlenecks [11, 14, 16, 21, 23, 46, 48, 56, 62, 65, 71]. Emerging *computational storage drives (CSDs)* present an in-storage processing model that shifts computation from the host to data-residing devices and thus eliminates data transfer overhead. CSDs further enhance performance by integrating application-specific

hardware accelerators as chiplets within storage controllers, while providing linear scalability through multi-CSD deployments that can scale processing capabilities proportionally with the number of attached drives.

The performance and efficiency benefits of CSDs have been demonstrated by extensive research across a diverse spectrum of applications, including high-performance computing [13, 41, 42, 61, 63], databases [15, 19, 20, 25, 32], data compression [49, 54, 68], graph computing [27, 34, 66], recommendation systems [33, 36, 37, 58], machine learning [24, 47, 60, 67], and emerging LLMs and RAG [16]. This academic momentum is mirrored by growing industrial adoption, with prominent commercial deployments such as Samsung's SmartSSD [52], the ScaleFlux CSD series [54], and their integration into large-scale systems like Alibaba's PolarDB [15]. To further solidify the ecosystem, the Storage Networking Industry Association (SNIA) and NVM Express (NVMe) have established foundational CSD architecture, programming model, and API standards [4, 43].

A major challenge that impedes the research and promotion of CSD technology is the lack of a proper CSD research platform. Existing platforms fall into hardware and software categories, as summarized in Table 1. **Hardware platforms**, such as OpenSSD [70], offer a full CSD system, which contains both CSD hardware (including compute and storage subsystems) and a software stack to access the CSD. Full-system research enables comprehensive examination and optimization of both concrete software and hardware components, facilitating empirical study and fine-tuning of their integrated behaviors. However, hardware platforms are *expensive* (e.g., a SmartSSD costs around $2,400 [24]), especially for multi-CSD configurations. Moreover, CSD configurations are *hardware-bounded*, where a next-generation CSD with computing capability higher than the hardware platform cannot be created.

Software platforms, which include emulation and simulation approaches, are *cost-effective* ($0) and run on commodity servers. **Software emulation platforms** model hardware on general-purpose servers for full-stack research without requiring physical devices. Emulation is straightforward when the modeled hardware configurations do not exceed the server's capabilities. For example, fast DRAM can effectively emulate slow flash storage by adding a timing delay model [35] However, emulating hardware accelerators in CSDs is fundamentally limited by server CPU's performance and efficiency. Hence, the configuration of computing capability with software-based CSD emulation is *CPU-bounded*.

Software simulation platforms use tools such as RTL (register-transfer level) synthesis and MQSim [59] to independently model the computational and I/O behaviors of CSDs [36, 42]. They can flexibly configure CSDs without hardware boundaries. However, they do not support full-system execution. Consequently, critical system-level behaviors and complexities may be omitted, leading to *inaccurate* performance evaluation. For instance, the simulation of offloading computation from an application to the CSD excludes important components like the system-level page cache, and our experiments on LevelDB in Section 6.2 reveal that CSD offloading leads to performance degradation unexpectedly due to cache bypassing. Existing CSD-based compression performance model assumes a constant compression latency [68], while our measurements show that actual compression latency varies with data patterns, ranging from 18μs to 25μs (a 38.8% deviation) for 4KB *Zstd* [18] compression on the host CPU. In addition, simulation approaches *do not support full-system design exploration*, since they cannot interact with real software or hardware.

Moreover, as shown in Table 1, current CSD research platforms are fragmented across diverse hardware and software simulators/emulators. This fragmentation compromises *compatibility*, hindering reproducibility and direct comparisons with prior work. In addition, the programming frameworks are customized and incompatible with conventional I/O stack. For example, λ-IO [70] and INSIDER [51] overload the Linux `pread` system call to offload computation, conflating compute and file read semantics in a single interface.

We present *CEMU*, a new open-source[1] CSD emulation platform designed to overcome the aforementioned limitations. It is completely software-based and allows full-system CSD studies at no cost. The well-established FEMU [35] SSD emulator is adopted as CEMU's storage subsystem, providing flexible and accurate storage emulation. CEMU overcomes two key challenges in the compute subsystem and software stack: **CPU-bounded CSD emulation** and **platform fragmentation**. To overcome the first challenge in emulating the CSD's compute subsystem, CEMU proposes a novel *VM-freezing* mechanism within the QEMU [6] *virtual machine (VM)* environment. It supports executing a CSD-offloaded task on the host CPU in a way that faithfully preserves its computational behavior and full-system functionality. Meanwhile, the application runtime clock within the VM can be temporarily frozen. Once the task is complete, the VM resumes operation after a configurable delay. From the application's perspective, the task is finished very fast as if the CSD has an extremely high-performance accelerator. Therefore, CEMU emulates a CSD with configurable computing capabilities through a computational timing model, enabling representation of FPGA- or ASIC-based accelerators that may outperform the host CPU.

To confront the second challenge, we employ two design principles. First, CEMU conforms to industry CSD standards, SNIA [43, 55] and NVMe [29], to address platform fragmentation. Second, CEMU is compatible with the existing I/O

[1] https://github.com/cs-qyzhang/CEMU

Table 1. Comparison of existing computational storage emulation platforms.

	Existing Work	CSD Platform	Full-System			Highly Configurable	Multi-Drive	Standard-Compliant	Open-Source
			Storage Subsystem	Compute Subsystem	Software Stack				
Hardware Platforms	λ-IO [70]	Daisy OpenSSD	✓✗	✓	✓	✗	✗	✗	✓
	Dstore [57]	ARM board+SATA HDD	✓✗	✓	✗	✗	✗	✗	✗
	BlueDBM [26]	Xilinx VC707 FPGA+flash board	✓	✓	✗	✗	✓	✗	✗
	GraphSSD [45]	OpenSSD+NAND flash	✓	✓	✗	✗	✗	✗	✗
	INSIDER [51]	AWS EC2 F1	✗	✓	✓	✗	✗	✗	✓
	Smart-Infinity [24]	SmartSSD	✓	✓	✓	✗	✓	✗	✓
	Summarizer [30]	ARM+FPGA+NAND flash	✓	✓	✓	✗	✗	✗	✗
Software Simulation	ECSSD [36]	Modified MQSim+RTL synthesis	✓✗	✓✗	✗	✓	✗	✗	✗
	ASSASIN [74]	GEM5+MQSim	✓✗	✓	✓✗	✓	✗	✗	✗
	DeepStore [40]	SCALE-Sim+SSDSim	✓✗	✓✗	✓✗	✓	✗	✗	✗
	MegIS [42]	Ramulator+MQSim+RTL synthesis	✓✗	✓✗	✗	✓	✗	✗	✗
Software Emulation	OmniCache [73]	Host CPU+Persistent Memory	✗	✓✗	✓✗	✗	✗	✗	✓
	blockNDP [12]	ARM QEMU+SSD	✓✗	✓✗	✓	✗	✗	✗	✓
	Wilcox et al. [69]	Host CPU+SSD	✓✗	✓✗	✓	✗	✗	✗	✗
	CEMU	**Emulated Compute+FEMU**	✓	✓	✓	✓	✓	✓	✓

stack, as CSDs are fundamentally storage devices. While the standards define CSD programming models and device interface, designing an operating system module that implements these interfaces, particularly in a way that is both user-friendly and compatible with the existing I/O stack, remains an open problem.

To this end, CEMU introduces a CSD-oriented software stack. The core is a newly developed file system, called *FDMFS*, which builds on the common use of file abstraction for storage access. FDMFS coexists with traditional file systems that access CSDs as regular SSDs, and provides a user-friendly programming framework for device memory management, computational task scheduling, and data access control through file system permissions in CSDs. In addition, FDMFS allows asynchronous operations and *peer-to-peer (P2P)* data transfer between CSDs to improve performance.

In summary, our core contributions are as follows.

- **CEMU**: a new open-source software platform that integrates free and highly configurable CSD emulation and a CSD-oriented software stack compatible with industry CSD standards and the traditional POSIX I/O stack. It enables accurate full-system and multi-CSD studies.
- **Platform validation**: CEMU has been validated against real CSD hardware, specifically Samsung SmartSSDs [52] and ScaleFlux CSD2000 [54]. Four synthetic benchmarks achieve an average emulation accuracy of 95%. Two full-system case studies, LLM training [24] and LevelDB [1], highlight CEMU's agility in prototyping CSD systems, requiring only a few hundred lines of code.
- **Insightful case studies**: By exploring different CSD configurations and examining full-system behaviors with CEMU, we can uncover insights that cannot be discovered in previous research platforms. The LLM training case study reveals that performance can be significantly improved by using multiple CSDs and ultrafast CSDs. However, a several hundredfold growth in CSD's computing power yields only a two-fold speedup, implying that a holistic system design is essential to realize the full potential of CSDs. The LevelDB case study illustrates potential side effects of CSD offloading, including host cache bypassing, I/O interference inside the CSD, and inter-CSD data transfer overhead. It suggests that we should deploy CSDs judiciously and manage the side effects with caution.

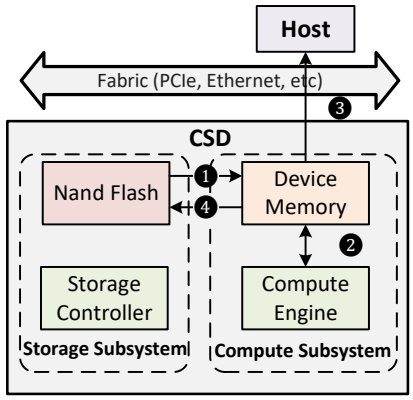


Figure 1. CSD internals.

2 Related Work and Motivation

2.1 CSD and Industry Standards

Figure 1 illustrates the internal architecture of a computational storage drive (CSD) [3, 4, 17, 36, 38, 43, 55]. A CSD comprises a storage subsystem typically based on persistent media such as NAND flash, and a compute subsystem that integrates a compute engine and device memory into the storage controller. Data is exchanged between these subsystems via an internal high-speed bus, enabling the offloading and execution of computational tasks from the host.

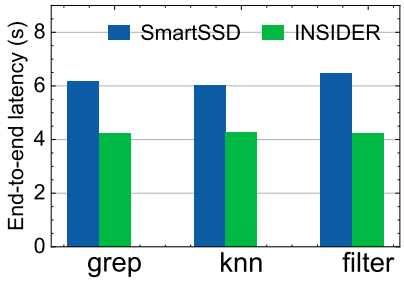


Figure 2. Latency comparison between INSIDER [51] and SmartSSD [52].

The industry has developed computational storage standards: the SNIA Computational Storage Architecture and Programming Model [43], which proposes two programming models, the *direct usage model* and *indirect usage model*; and the NVMe Computational Programs Specification [4], which leverages NVMe as the interface for CSDs.

Programming model. As shown in Figure 1, performing a computational task in the CSD comprises three steps: 1) transferring data from NAND flash to device memory ❶; 2) executing computation in device memory ❷; and 3) reading output data from device memory to the host ❸ or writing it to NAND flash ❹. In the direct usage model, the three steps are explicitly initiated and scheduled by the host. By contrast, the indirect usage model embeds the computational flow within an IO request, which the host sends to the CSD for automatic processing along the IO path.

NVMe interface. NVMe introduces two new namespaces for CSDs: the *compute namespace* and the *memory namespace*. The compute namespace is designed for offloading and executing computational tasks via *load* and *execute* commands, respectively. The memory namespace is used for managing device memory with three main commands: memory copy, memory read, and memory write. The memory copy command facilitates data transfer between the conventional flash storage namespace and the memory namespace.

2.2 Existing CSD Platforms

Table 1 summarizes existing representative hardware- and software-based CSD platforms. They are compared in the dimensions of full-system support, multi-CSD scalability, standard compliance, and open source.

Full-system support. Although real hardware platforms allow full-system research, such as Samsung SmartSSD or FPGA boards, they are not cheap to acquire, require non-trivial development efforts, and lack configurability to explore different CSD configurations. For example, λ-IO [70] uses a specific hardware platform OpenSSD (with an outdated PCIe Gen3 interface) [31], not available to the authors of the following work OmniCache [73]; thus, they reproduce λ-IO via software emulation. INSIDER [51] employs a Xilinx's UltraScale+ series FPGA to create the CSD compute subsystem. However, the FPGA does not include flash storage, and a simplified storage subsystem is emulated in DRAM by adding constant I/O delays. As flash parallel architectures and operational behaviors are not faithfully emulated, the emulation accuracy degrades. Figure 2 compares INSIDER with a SmartSSD that integrates the same FPGA under three computational tasks. The results reveal a notable end-to-end latency gap with a 34% error rate.

Software emulation/simulation presents a promising avenue for exploring the design space of CSD systems. For simulation platforms, they model computational and I/O delays of CSDs by combining a processor/accelerator simulator as the compute subsystem (e.g., RTL analysis) and an SSD simulator as the storage subsystem (e.g., MQSim [59]). However, they cannot interact with actual applications and hardware components and do not involve realistic data processing. Thus, they miss full-system behaviors and degrade end-to-end emulation accuracy.

Although `gem5` [39] offers a full-system mode, it is primarily intended for micro-architecture simulation, rather than systems research. It performs single-threaded clock-driven simulation, which is very slow, such as taking five hours for tens of seconds of SSD simulation [22]. Additionally, blockNDP [12] and Wilcox's CSD emulator [69] employ QEMU-based full-system emulation but are limited to functional emulation of CSD offloading. They lack performance modeling and the CSD's computing capability configuration is bounded by the host CPU. In contrast, our CEMU is developed from FEMU [35], a widely used SSD emulator for full-stack systems research. We augment it with a highly configurable compute subsystem (§3.2).

In addition, most prior platforms either lack a software stack or employ ad hoc programming frameworks that are incompatible with the current I/O stack. We design a CSD-oriented software stack compatible with industry standards and the existing I/O stack, allowing users to invoke Linux system calls with faithful semantics to access CSD computational resources, such as `copy_file_range` to load data from flash storage to CSD device memory for processing and `pread` to read result from the CSD to the host.

Multi-CSD scalability. Most existing CSD research does not investigate multi-CSD systems, while Smart-Infinity [24] and BlueDBM [26] construct multi-CSD hardware platforms that have high costs and poor availability. The former implements a hardware platform with 10 SmartSSDs and a chassis for PCIe slot expansion. The latter emulates multiple CSDs by integrating FPGA and NAND flash cards across multiple servers. Our CEMU allows the creation of multiple CSD instances in a general server and improves data transfer efficiency with P2P communication between CSDs.

Standard compliance and open source. None of the existing CSD platforms conforms to the SNIA and NVMe industry standards, and most of the platforms are not open source. As a result, the CSD research landscape is fragmented,

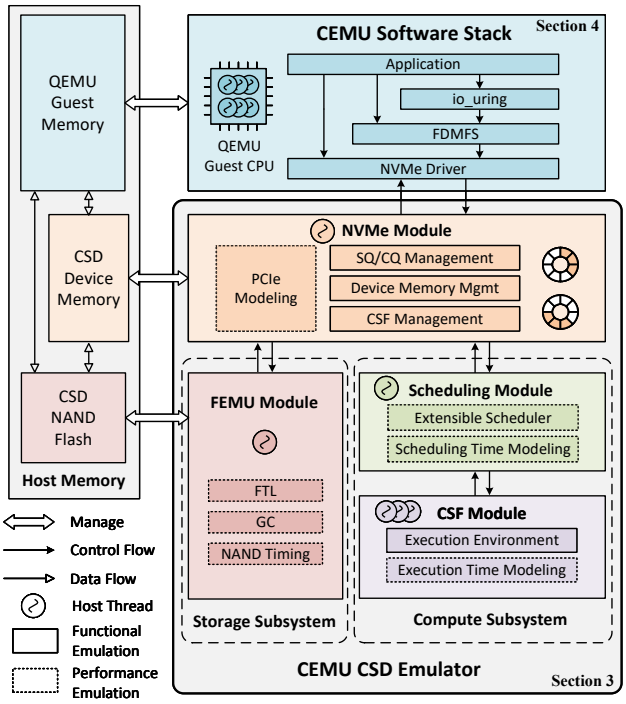


Figure 3. Overall architecture of CEMU.

which causes significant difficulties in reproducing previous work and advancing the research. In developing CEMU, we emphasize standard compliance and aim to provide an open-source generic CSD emulation platform.

3 CEMU CSD Emulator

3.1 Overview of CEMU

We design and implement CEMU, the first full-system CSD emulation platform that integrates a highly configurable CSD emulator and a CSD-oriented software stack compatible with industry CSD standards and the current I/O stack. CEMU is developed based on QEMU [6], which allows the CSD emulator to be mounted as a standard NVMe drive within the guest VM. Figure 3 illustrates the overall architecture. The CSD emulator has four modules: the NVMe module, the CSF module, the scheduling module, and the FEMU module. CEMU CSD exposes three NVMe namespaces: the conventional NVM storage namespace, CSD-oriented compute namespace and memory namespace.[2] The latter two are used to execute computational tasks and manage device memory (see §2.1).

The CSF module offers execution environments for computational tasks, emulating functional behaviors and timing models. The scheduling module manages tasks and enables the development of new scheduling algorithms. Together, the CSF and scheduling modules form the CSD's compute subsystem (§3.2). The FEMU module adopts the widely used FEMU SSD emulator as the CEMU storage subsystem (§3.3). For multi-CSD support, CEMU addresses FEMU's scalability limitations by supporting PCIe *peer-to-peer (P2P)* communication between CSDs. CEMU also includes a complete CSD software stack, which will be introduced in Section 4.

[2]The FEMU module and the SQ/CQ management within the NVMe module are modified based on the FEMU codebase, and the other components are newly developed. *Computational storage function (CSF)* is a term defined in SNIA Computational Storage Architecture and Programming Model [43] for the programs or tasks offloaded to run inside the CSD.

3.2 CSD Compute Subsystem

3.2.1 CSF Execution Environments. To improve flexibility and allow researchers to explore various execution environments, CEMU provides three CSF types for offloading tasks into CSDs. (1) **extended Berkeley Packet Filter (eBPF)** [8] is a revolutionary technology for running sandboxed programs in a privileged environment. Due to good security and comprehensive toolchains, eBPF is a promising CSF format for CSDs. CEMU utilizes ubpf [7] to run eBPF programs on the host CPU. (2) CEMU can use **shared libraries** (`.so` files) as the CSF to support OS and Docker execution environments. This enables users to easily migrate existing applications to CEMU CSD. At runtime, CEMU leverages *libdl* [5] to dynamically load and execute shared library-type CSF on the host CPU. (3) CEMU also supports the **bitstream**-type CSF when an FPGA card is used as the CSD compute engine. The user can generate a bitstream file that implements the CSF and download it to the FPGA to execute.

3.2.2 Computational Performance Modeling. CEMU can utilize the host CPU (by default) or an FPGA card (if available) to physically execute offloaded CSFs within CSDs. As a cheap and highly configurable CSD emulator, CEMU must support various computing capability configurations, not constrained by the host CPU or FPGA. To this end, CEMU enables modeling computational time based on real CSF execution time. We use *compute unit (CU)* to abstract a processing core. The response time ($T_{response}$) of a computational task running in a CU is calculated as the sum of T_{exec} and T_{sched}. T_{exec} represents the execution time of the task, while T_{sched} denotes the queuing or waiting time in task scheduling.

***T_{exec}* modeling.** CEMU offers a scaling factor parameter S_{csf} that can be configured by the user. It applies to scale the actual execution time of the task on the compute hardware (e.g., host CPU), denoted as T_{host}, that is, $T_{exec} = S_{csf} \times T_{host}$ (S_{csf} is the scale factor). When a CSD with a compute engine (e.g., an ARM CPU) slower than the physical compute hardware is emulated, CEMU can use real execution time by throttling the computation speed, such as lowering the CPU frequency, or model the execution time by setting $S_{csf} > 1$. Conversely, if CEMU emulates a CSD with a compute engine faster than the physical compute hardware (e.g., a hypothetical ASIC accelerator), performance modeling is required by configuring $S_{csf} < 1$. As there may be no real hardware available to calibrate S_{csf}, we can utilize existing cycle-accurate simulators like `gem5` to model the accelerator

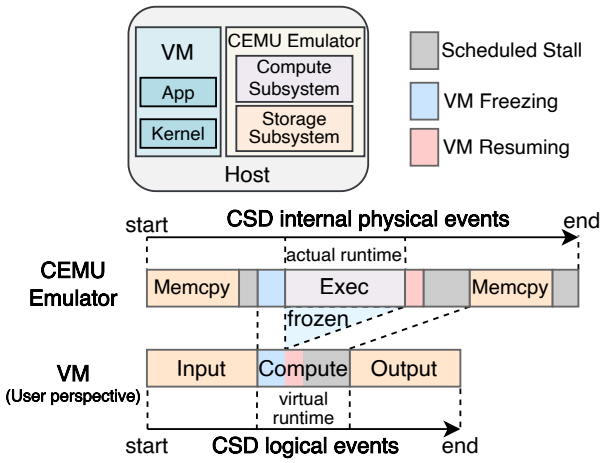


Figure 4. Timeline of CEMU's internal physical events and the VM user perspective.

and obtain its compute capability, or a range of scale factors can be configured to model potential accelerators.

Emulating such a fast compute engine is challenging because it would require returning results before the task has been completed on the physical compute hardware. To address this challenge, CEMU employs a VM-freezing mechanism, which is introduced in §3.2.3.

***T_{sched}* modeling.** The T_{sched} of a computational task depends on scheduling. CEMU contains a pluggable scheduling module that allows the user to implement different scheduling policies. Specifically, four hook functions are provided: `enqueue_job()`, `dequeue_job()`, `pick_next_job()`, and `switched_out()`. CEMU includes first-in-first-out and round-robin scheduling policies by default.

Multiple CUs. CEMU can configure multiple CUs to emulate multiple compute engines in CSDs. Each CSF can be bound to specific CUs, akin to the CPU core affinity in Linux. Furthermore, each CU can emulate a distinct type of compute engine. For instance, one CU may be designated to emulate a compression accelerator, while another serves as a general-purpose core. In multi-CU emulations, each CU can be assigned an individual scaling factor and a distinct scheduling policy, enabling fine-grained control over heterogeneous computing emulation.

Device Memory. For eBPF- and shared-library-type CSFs, CEMU allocates device memory in host DRAM, whereas bitstream-type CSFs utilize FPGA memory. Device memory management (e.g. alloc, free, and permission control) is handled by the CSD-oriented file system, FDMFS, which will be discussed in Section 4.

In summary, the CEMU CSD enables flexible configurations of the compute subsystem, including the compute capability of each CU, the number of CUs, the device memory size, and the scheduling policy.

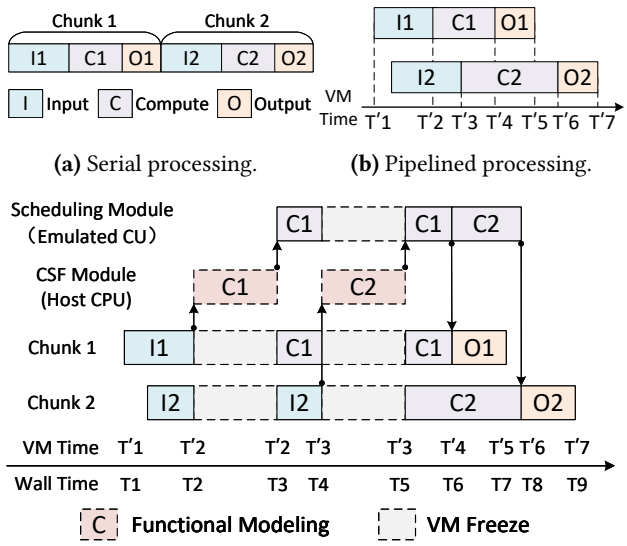
3.2.3 *High Configurability with VM-Freezing.*

CEMU performs CSF computation on the host CPU (if not using an external FPGA card). To emulate a compute engine exceeding the performance of the host CPU, we propose a **VM-freezing mechanism**. In CEMU, user applications run inside a QEMU VM, while CSD emulation executes on the host CPU. When input data have been loaded into the device memory and the CSF task is initialized, the VM can be paused, where the virtual CPU (vCPU) and its clock are stopped. The task continues to run on the host CPU until its completion. This delay is not observable within the VM. The VM is then restored, making the task appear instantaneously completed from its view. Therefore, the CEMU scheduler can add a *stall time* to emulate the task execution time overhead, which is calculated based on the T_{exec} modeling (see Section 3.2.2). The timeline is illustrated in Figure 4. From VM perspective, the task execution time is the configurable stall time plus the time overheads of VM pause and resume operations. It can be smaller than the actual time that the task is executed on the host CPU.

While VM freezing conceals the actual computation delay, VM pausing and resuming operations introduce VM-detectable latency overheads. To handle a VM pausing request, the QEMU main thread first acquires the Big QEMU Lock (BQL), then stops the global VM clock, and subsequently broadcasts a pause request to all vCPUs (through `pause_all_vcpus()` and `cpu_disable_ticks()` in QEMU). After reaching a safe execution point, every vCPU acknowledges the request and sleeps. The VM resuming operation follows a symmetric control flow. The QEMU main thread re-enables the global VM clock and broadcasts a resume request to all vCPUs (via `resume_all_vcpus()` and `cpu_enable_ticks()`). Each vCPU wakes up and continues execution from the same state as before the pause. The entire procedure incurs about 21μs of latency on our server. In addition, fine-grained per-vCPU freezing is not supported in QEMU, since the global VM clock pauses. Minor variations in the exact suspension point of different vCPUs do not affect the correctness or timing observability, as no guest execution progresses while the VM clock is paused.

The 21μs latency overhead imposes a minimum task execution time for VM-freezing-based modeling. Considering a computational task with 4MB input data, the maximum CSF processing throughput is 184GB/s in one CU. The throughput can be multiplied by setting up multiple CUs for parallel processing and reducing the frequency of VM freezing, such as grouping multiple tasks for batch processing and increasing the data chunk size. If the application offloads sub-21μs computational tasks, the CSD scheduler automatically merges the tasks as much as possible in each VM freezing operation.

The VM-freezing mechanism supports pipelined chunk processing in one CU and concurrent chunk processing in multiple CUs or CSDs. The processing of a chunk includes



(a) Serial processing. **(b)** Pipelined processing.

(c) Pipelined processing details of two chunks in one CU.

Figure 5. Pipelined processing of chunks in the CSD.

three phases: data input, computation, and result output. In the serial processing model, data chunks are processed sequentially in one CU, where the CU is idle in the input and output phases (see Figure 5a). To improve CU utilization, the chunks can be processed in a pipelined manner. Figure 5b shows the logical view, while Figure 5c depicts the pipelined processing details of two chunks in one CU of a CSD. The CU is emulated using the host CPU, where the computation physically occurs. At time T_2' (T_2) when the input of *Chunk 1* is completed, its computation starts and the VM is frozen. After *Chunk 1* has finished computation at time T_3, the VM is resumed. For *Chunk 2*, its input phase is paused at T_2, resumed at T_3, and ended at T_4. The VM is frozen again and then restored at T_5, during which the computation of *Chunk 2* is performed. Once the computation of *Chunk 1* or *Chunk 2* is completed on the host CPU, the CSD scheduler adds a stall time to model the computation time in the emulated CU. After the computation emulation, the output operation is executed. If the system contains multiple CSDs or a CSD with multiple CUs, multiple chunks can be processed in parallel. For example, the computation of *Chunk 2* can be emulated in another CU and begin immediately at T_3' (T_5).

3.3 CSD Storage Subsystem

3.3.1 FEMU Module. We employ FEMU [35], a widely used SSD emulator, to model the storage subsystem.[3] FEMU emulates flash storage with a parallel architecture and faithful operational behaviors in host memory. A *flash translation layer (FTL)* is incorporated to manage flash storage. It runs in a separate thread to avoid resource contention between the compute and storage subsystems.

We enhance FEMU with a PCIe bus model that adds data transfer overhead, which is important because a primary goal of computational storage is to reduce data transfer. Specifically, the PCIe modeling simulates transmission queues and, based on the configured bandwidth, adds calculated queuing and propagation delays to each I/O request.

3.3.2 Multiple CSDs. To emulate a multi-CSD system, the compute subsystem must be scalable. CEMU assigns dedicated CPU cores and memory resources to each CSD. However, QEMU requires the BQL during VM pause/resume operations, which can hinder scalability. CEMU reduces the lock overhead by using atomic reference counting, enabling lightweight VM freezing.

Data transfer overhead between multiple CSDs becomes a concern when the working set is distributed across multiple CSDs. A straightforward approach is to rely on the host CPU to read data from one CSD to host memory and then write the data to another CSD. This approach consumes the host CPU and memory resources and has a long transfer path, degrading system performance and scalability. To reduce this overhead, CEMU leverages PCIe P2P, enabling direct transfers between CSDs via a PCIe switch. We implement PCIe P2P transfer in CEMU by exposing each CSD's device memory through PCIe Base Address Registers (BARs). During PCIe enumeration, the host assigns a unique bus address to each CSD's BAR space. Inter-CSD data transfer is achieved with a single P2P I/O operation, where the destination is set to a remote CSD's BAR address.

The scalability of CEMU is restricted by the CPU and memory resources of the host server, as QEMU cannot create a VM across multiple hosts. CEMU emulates flash storage in host memory. The aggregate capacity and I/O bandwidth of multiple CSDs cannot exceed the capacity and bandwidth of the memory. The emulation of each CSD consumes four CPU cores. The original FEMU uses two cores to run the SSD FTL and the NVMe module, while CEMU requires two more cores for the scheduling and CSF modules (see Figure 3). A server generally carries hundreds of GBs to a few TBs of memory, with bandwidth reaching several hundred GB/s, and tens of CPU cores. These resources are usually sufficient to prototype CSD systems for validating research ideas. For example, 32 CPU cores and 800GB memory can be allocated to emulate an array of 8 CSDs, each with 100GB storage capacity and 10GB/s I/O bandwidth.

It is possible to expand the scalability of CEMU beyond a single host. First, a distributed application can create multiple QEMU instances across different hosts, with each instance holding several CSDs. A computational task and its dataset

[3]We do not choose SSD simulators like MQSim [59] because they do not support full-system emulation with realistic data processing. NVMeVirt [28] is a recently proposed SSD emulator. CEMU does not employ it due to inflexibility for CSD emulation. Implemented as a host kernel module, NVMeVirt cannot support VM freezing to model advanced compute engines and faces constraints, such as on linking external operator libraries.

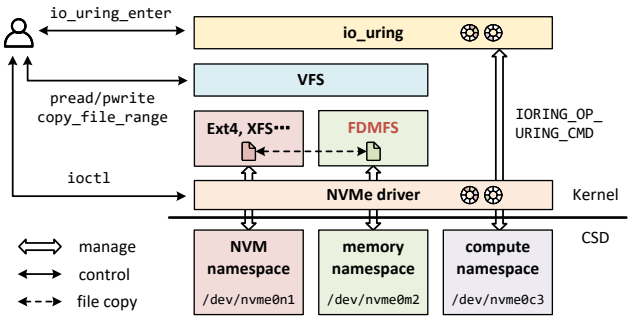


Figure 6. CSD-oriented software stack of CEMU.

can be divided and assigned to these CSDs for parallel processing. Second, CXL- or RDMA-connected remote memory [44] can be employed to extend host memory and thus the aggregate storage capacity of emulated CSDs. In addition, it is feasible to use a real SSD to break host memory capacity limitation for emulating CSD storage. In this case, a VM-freezing operation needs to be scheduled for processing an I/O request when the SSD access latency exceeds the modeled I/O latency.

4 CEMU Software Stack

4.1 Design of FDMFS

A CSD-oriented software stack should adhere to industry standards and offer good usability. It should also maintain compatibility with the existing I/O stack, since CSDs retain the storage functionality of SSDs. Modern I/O stacks are predominantly structured around file systems. Hence, the CEMU CSD software stack utilizes file abstraction to incorporate the offloading, scheduling, and execution of computational tasks. To this end, we introduce a new CSD-oriented file system, called **FDMFS**.[4]

Device memory is a key component in a CSD, for example, all four operations in the data processing path (Figure 1) involve it. FDMFS abstracts device memory as files, allowing users to manage it with familiar file interfaces, such as `fallocate` for memory allocation, `pread/pwrite` for memory read/write operations, and file permissions for data access control. Section 4.2 describes how these system calls are used to complete a computational task in a CSD.

Based on FDMFS, we develop a complete CSD software stack depicted in Figure 6. The bottom is the NVMe device driver, which is modified to adapt to the NVMe Computational Programs Specification [4]. This standard introduces the memory namespace and the compute namespace, along with corresponding sets of NVMe commands such as memory read/write, program download and execution. To integrate with the existing I/O stack, we add a block device in the kernel to access the memory namespace and a character device to access the compute namespace. FDMFS runs on top of the block device, and file creation and deletion refer to memory space allocation and release, respectively.

[4]FDM stands for "Functional Data Memory", a term defined in SNIA Computational Storage Architecture and Programming Model [43] for the device memory in CSDs.

```
ctrl_fd = open("/dev/nvme0c3", O_RDWR);
nvm_fd = open("/ext4/data", O_RDWR | O_DIRECT);
csf_fd = open("/ext4/program", O_RDONLY);
in_fd = open("/fdmfs/input", O_RDWR | O_DIRECT);
out_fd = open("/fdmfs/output", O_RDWR | O_DIRECT);
// download
csf_id = ioctl(ctrl_fd, IOCTL_DOWNLOAD, csf_fd);
// read input data
copy_file_range(nvm_fd, in_fd);
// compute
ioctl(ctrl_fd, IOCTL_EXECUTE, in_fd, out_fd, csf_id);
// read output data
data = pread(out_fd);
```

(a) Direct usage model.

```
...
// indirect task
data = ioctl(ctrl_fd, IOCTL_EXECUTE, in_fd, out_fd,
     ↪ csf_id, indirect=1, concurrent_chunks=1);
```

(b) Indirect usage model.

Figure 7. Pseudo-code used by the CEMU software stack.

Managing device memory through a file system requires careful consideration of file mappings. In a conventional storage-oriented file system like `ext4`, a file's mapping contains scattered extents. However, if non-contiguous device memory regions are mapped to an FDMFS file, a *memory management unit (MMU)* would be needed within the CSD, which would introduce a hardware limitation. To address this constraint, CEMU requires programs to pre-allocate the target memory-file size using `fallocate()`. Then, a contiguous memory region can be assigned and mapped. To resize the memory mapped to an FDMFS file, CEMU deletes the old file and creates a new one in the current implementation. It is possible to perform dynamic memory resizing on the fly with non-contiguous memory allocation. The NVMe CSD specification has defined a memory range set, which can describe multiple non-contiguous memory regions. CEMU can map an FDMFS file to a memory range set and resize memory allocation by dynamically adding or removing memory ranges. We leave this extension as future work.

For the compute namespace, CEMU uses `ioctl` system calls for the downloading and execution of CSF tasks. Figure 7 exhibits an example how system calls are used under different usage models (introduced in the next section). The compute namespace allows CEMU to easily manage a computational task as an I/O task, leveraging the existing I/O stack such as `io_uring` for task management.

4.2 Standard Programming Models

CEMU software stack supports two programming models for CSDs, the *direct usage model* and the *indirect usage model*, which are standardized by SNIA (§2.1).

Direct usage model. In this model, the execution process of a computational task is explicitly controlled by the host, including data input, computation, and output of the result.

The user first allocates input and output device memory space for execution by creating FDMFS files. Then, data is transferred from a data file (e.g., an ext4 file) to the input memory file. Simply using `pread` on the data file followed by `pwrite` on the memory file would be inefficient, as it would cause the data to flow through the host. To address this, CEMU utilizes the Linux system call `copy_file_range`. It copies data directly from a file to another, avoiding the context switch and data copy overheads between the kernel space and user space. With FDMFS, CEMU can use `copy_file_range` to move data directly between in-drive flash and device memory without going through the host. Specifically, when receiving a `copy_file_range` call, FDMFS retrieves the mappings of the data file and the memory range of the memory file, and issues an NVMe memory copy command to the CSD.

To initiate a computational task, an ioctl request is submitted to the compute namespace. After the computation finishes, the results are kept in the output memory file. They are either written back to a data file in flash via `copy_file_range()` or read to the host using `pread` on the output memory file. Figure 7a shows a code example that employs the direct usage model to offload a computational task, read input data, execute the task, and read output data in the CSD. In addition, `copy_file_range` can also be leveraged to carry out P2P transfers between CSDs by setting `nvm_fd` and `in_fd` (line 9 in the figure) to the corresponding files in the source and destination CSDs.

Indirect usage model. In this model, the host only submits a computational request through the `ioctl` system call, as shown in Figure 7b, while the CSD firmware manages the whole task execution process. Therefore, the indirect usage model can avoid frequent host-CSD communication and synchronization.

Comparison of direct and indirect usage models. The direct usage model is host-centric, since the host initiates each step to proceed with a computational task in the CSD. It is suitable for complex scenarios where input data may be distributed across multiple files or be dependent on the output of another task. In contrast, the indirect usage model is device-centric, where the CSD automatically performs an offloaded computational task. This model avoids frequent host-CSD communication and synchronization and simplifies the development of applications.

Asynchronous Operations. As CEMU software stack is integrated with the existing I/O stack, operations on device memory and computation can take advantage of the current asynchronous I/O framework to boost performance. In CEMU, we utilize `io_uring` [9] to perform asynchronous operations in all three CSD namespaces. In the direct usage model, the host program must explicitly manage the synchronization between operations, such as submitting the compute command after the read command that loads data to the device memory has been completed. In the indirect usage model, the CSD firmware is responsible for scheduling and synchronizing the operations.

Table 2. Hardware Configurations.

Test Server	
CPU	Intel Xeon Platinum 8458P @2.7GHz
Memory	1TB DDR5 @4800Mbps
CSD	3 × SmartSSD @PCIe3.0 × 4
SmartSSD	
Bandwidth	3GB/s (external), 2.5GB/s (internal)
Compute subsystem	Kintex Ultrascale+ KU15P @300MHz FPGA
Device memory	4GB DDR4 @2400Mbps
CSD2000	
Bandwidth	3GB/s (read), 2.3GB/s (write)
Compute subsystem	Xilinx Ultrascale+ FPGA
CEMU CSD	
Bandwidth	Consistent with SmartSSD/CSD2000
Flash Storage	8 channels, 6 dies per channel

5 Evaluation

In this section, we evaluate CEMU comprehensively, including hardware validation, software stack analysis, scalability, and full-system case studies on LLM training and LevelDB.

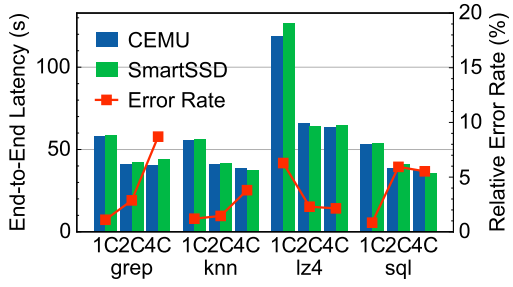
5.1 Experimental Setup

We implement CEMU based on QEMU 8.0.0 and Linux 6.8. The emulator part is implemented based on the FEMU codebase [2], with added functional and performance modeling of the in-drive computing subsystem. The software stack is integrated into the Linux kernel, incorporating modifications to the NVMe driver, the FDMFS file system, and the `IORING_OP_COPY_FILE_RANGE` operation in `io_uring`. The entire implementation comprises about 11K *lines of code (LoC)*, including 8K lines in QEMU and 3K lines in the kernel.

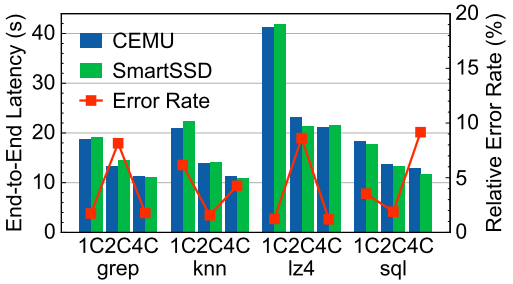
Industrial CSD hardware platforms are used to validate CEMU, including Samsung SmartSSD [52] and ScaleFlux CSD2000 [3]. Table 2 presents the configurations of the test server and CEMU CSD, and the key parameters of SmartSSD and CSD2000.

5.2 CEMU CSD Emulator Validation

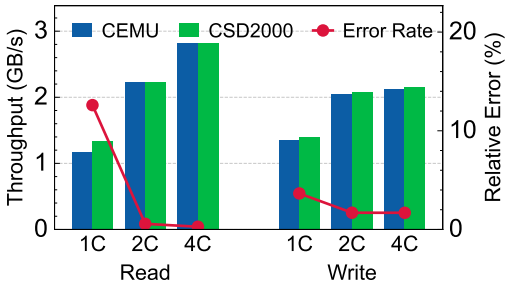
We compare CEMU with two real CSDs, SmartSSD and compression-enabled CSD2000, to evaluate its accuracy. For SmartSSD, we select four representative compute functions: *grep*, *kNN*, *LZ4 compression (LZ4)*, and *TPC-H query 1 (sql)*. These functions are implemented on SmartSSD using Xilinx Vitis HLS (high-level synthesis), based on the INSIDER



(a) A single CSD.



(b) Three CSDs.



(c) Bandwidth of compression-enabled CSDs.

Figure 8. End-to-end performance validation. '1C/2C/4C' stands for one, two, or four chunks (processed in parallel).

codebase [51] and the Vitis_Libraries [10] repository. We execute each compute function on both the host CPU and SmartSSD/CSD2000, obtaining T_{host} and T_{exec}, respectively. The ratio of these values is then used to derive S_{csf} (Section 3.2.2). Each test is conducted on a 100GB dataset and processed in 16MB chunks.

Figure 8 presents the end-to-end latency or throughput, and relative error rates of CEMU, CSD2000, and SmartSSD across different pipeline chunk configurations and varying numbers of CSDs. For the three-CSD setup, we uniformly distribute the dataset across the three drives. Compared to hardware platforms, CEMU maintains an error rate below 10%. On average, CEMU achieves a high accuracy of 96% in the single-SmartSSD emulation (Figure 8a), 95% in the three-SmartSSD emulation (Figure 8b), and 97% in the CSD2000 emulation (Figure 8c). In addition, the results suggest that concurrency of two chunks is sufficient to obtain the maximum performance in most cases.

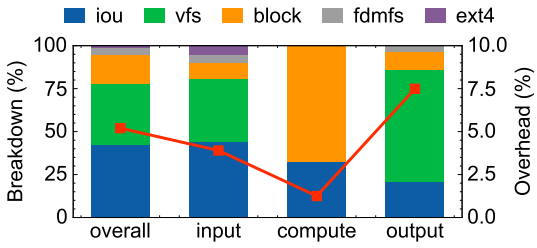


Figure 9. Software overhead breakdown.

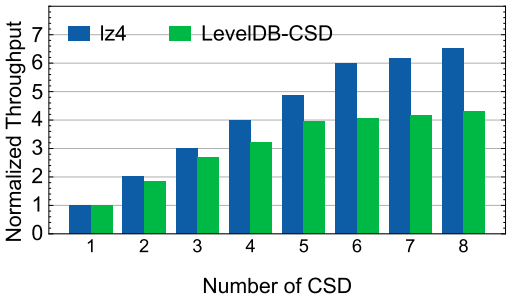


Figure 10. CEMU scalability with up to eight CSDs.

5.3 CEMU Software Stack Analysis

5.3.1 Overhead Analysis. We use the LZ4 application to profile the overhead of CEMU's software stack. To accurately record the overhead, we perform computation sequentially with a single chunk. As there is only one task submission/-completion in the indirect usage model, the software stack overhead can be neglected. Only the direct usage model is tested. Figure 9 shows the overhead breakdown analysis in the software stack, including the `io_uring`, `vfs`, `block layer`, `FDMFS`, and `ext4`. We can see that the software overhead does not exceed 7.5% in the overall task completion time and in the execution time of each phase (input, computation, and output). Among the different phases, the output phase has the highest overhead, as it reads data from the device memory to the host. It is also noted that FDMFS is lightweight and causes negligible software overhead.

5.3.2 Programming Models. To understand the efficiency of the two usage models, we perform tests on LZ4 with a data set size of 1GB. The tests configure different numbers of pipelined chunks and different chunk sizes. With a single chunk, the indirect usage model outperforms the direct usage model, with a performance difference of 7% at a chunk size of 1MB. This is because the indirect usage model avoids the software stack overhead caused by frequent synchronization for each chunk computation. As the chunk size increases, the performance gap between the indirect and direct usage models gradually decreases, from 7% at 1MB to 4% at 2MB and 3% at 4MB. In addition, the two models have similar performance under multiple chunks, as pipelining the processing of chunks alleviates the synchronization overhead.

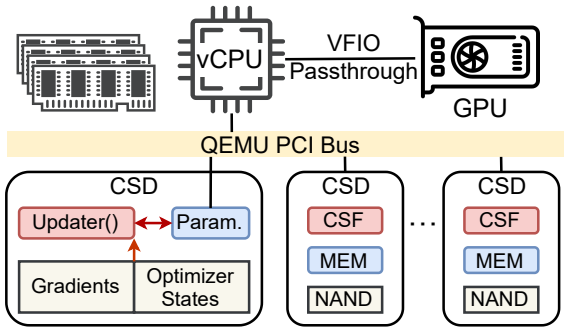


Figure 11. CEMU environment in Smart-infinity.

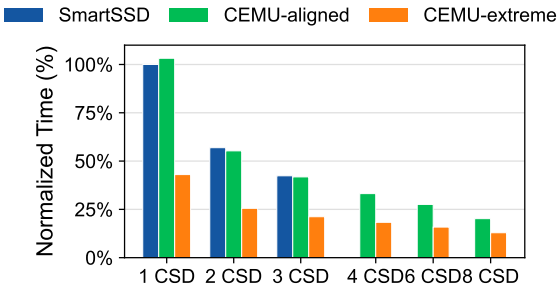


Figure 12. Training time of Smart-Infinity.

5.4 Scalability of CEMU

We use LZ4 compression and LevelDB, a popular key value store, to evaluate CEMU scalability. For LZ4, we distribute the data evenly across multiple CSDs and perform compression simultaneously in each CSD. For LevelDB, its compaction tasks are offloaded into the CSD through a shared library-type CSF, as detailed in Section 6.2. A separate LevelDB instance is launched on each CSD. Figure 10 shows the normalized aggregated throughput when the number of CSDs increases from 1 to 8. We can see that scalability is high in LZ4, where the throughput scales almost linearly and reaches the limit at six CSDs. In contrast, the throughput increases slower and saturates earlier in LevelDB. Although compaction is offloaded, LevelDB still performs complicated data processing on the host side, which becomes the performance bottleneck. These results demonstrate the benefits of CSD technology in scalability and the effectiveness of CEMU in multi-CSD studies.

6 Full-System Case Studies

Recent studies have explored CSD technology to accelerate LLM training [24, 50] and LSM tree-based key-value stores [53, 72]. We utilize these two representative applications to showcase CEMU's efficiency in full-system research. We not only validate CEMU's emulation accuracy with SmartSSDs at the system level, but also provide a comprehensive understanding of CSD systems by investigating the impacts of different computing capabilities in CSDs, host-side caching, CSD-internal I/O interference, and inter-CSD P2P transfer on system performance. The investigation yields valuable insights that cannot be discovered in previous research platforms and outlines promising directions for future research.

6.1 Large Language Model Training

The rapid advancement of LLM has been driven by the increase in the number of parameters, leading to excessive capacity requirements for GPU memory during training. To alleviate the problem, recent studies have explored storage offloading techniques that utilize SSDs as extended memory [50]. In particular, Smart-Infinity [24] leverages CSDs to address the storage bandwidth bottleneck. It offloads parameter updates to FPGA-based accelerators in SmartSSDs, reducing host-storage data transfer by 75%. We port Smart-Infinity to CEMU CSDs, as shown in Figure 11. GPU is passed through to the VM through VFIO, which can achieve near-bare-metal performance [64]. The implementation requires only ~200 LoC modifications. This highlights the agility of CEMU to prototype CSD systems and validate CSD research concepts, which may take weeks to develop on FPGA platforms.

Two types of CEMU CSDs are evaluated. *CEMU-aligned* CSDs are configured to emulate SmartSSDs, where the compute bandwidth in each CSD is around 1.25GB/s. To demonstrate the high configurability of CEMU, we also set up *CEMU-extreme* CSDs, where the CSD compute bandwidth is scaled to 700GB/s with the VM-freezing-based modeling. The experiments train the GPT-2 (1.7B parameter) with a batch size of 4. The gradients and optimizer states files occupy roughly 37GB. The updater runs inside the CSD and consumes 4GB device memory to store half-precision parameters, gradients, and optimizer states. After each computation cycle, the host loads the updated parameters and writes the results to relevant files.

Figure 12 shows the LLM training time under SmartSSDs, CEMU-aligned CSDs, and CEMU-extreme CSDs. The number of CSDs ranges from 1 to 8, while the number of available SmartSSDs is 3. When the number of CSDs increases, the LLM training time decreases significantly. For example, the speedup is 2× to 2.5× when the number of CSDs grows from 1 to 3. CEMU-aligned CSDs demonstrate high emulation accuracy with an average deviation of just 2.4%, compared to SmartSSDs. Compared to CEMU-aligned CSDs, CEMU-extreme CSDs deliver up to 2.4× speedup, despite having compute engines that are hundreds of times faster. The improvement is small mainly because the I/O overhead becomes the system bottleneck.

From these results, we can derive three conclusions and insights. *First, CEMU provides an agile and highly configurable emulation platform for cutting-edge CSD system research. Second, CSD systems have high scalability, as storage and computing capacities grow linearly with the number of*

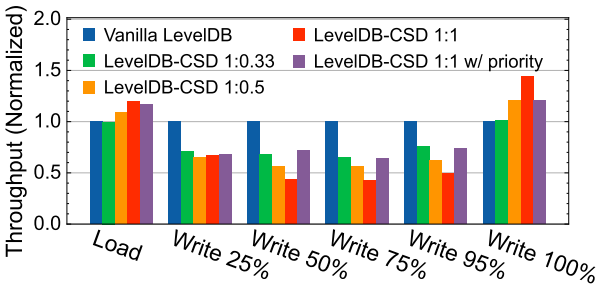


Figure 13. Normalized throughput of LevelDB-CSD compared to Vanilla LevelDB under different write ratios.

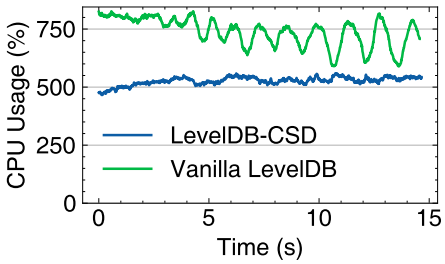


Figure 14. CPU usage of 5 instances on 5 drives.

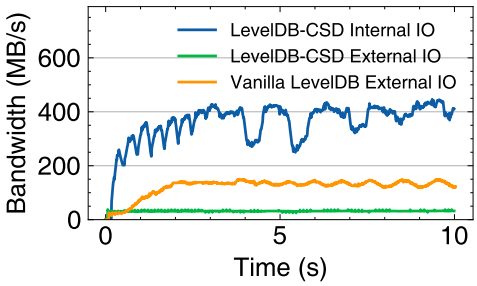


Figure 15. I/O bandwidth in load phase.

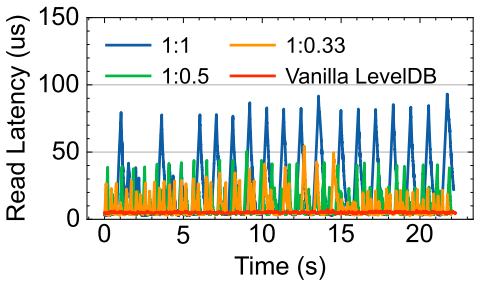


Figure 16. Latency of read operations.

CSDs. Third, enhancing the computing power of CSDs alone may be inefficient in improving system performance. To realize the full potential of CSDs, a holistic approach is essential to balance I/O and computing efficiencies.

6.2 LevelDB

Offloading LSM-tree compaction into CSDs. Popular key-value stores, such as LevelDB and RocksDB, employ the LSM tree as the fundamental data structure. LSM trees store key-value pairs in *sorted strings table (SSTables)* organized in multiple levels. A compaction routine is performed periodically to merge the SSTables between levels to balance write throughput and read performance. Compaction operations cause significant CPU and I/O overheads. To address the problem, we offload LevelDB's background compaction into the CSD via a shared library-type CSF. This scheme is referred to as *LevelDB-CSD*. Based on the user-friendly CEMU software stack, the implementation of LevelDB-CSD requires only around 500 LoC.

CSD Acceleration. Figure 13 shows the throughput under varying read-write ratios and different configurations of CSDs. The legend $1:x$ indicates that the ratio between the CSD's computing power and the host CPU's is x (a higher x means that the CSD has a faster compute engine). The workload uses 8-byte keys and 16-byte values, and initially loads 10 million records followed by another 10 million read-/write requests. In the 100% write workload, LevelDB-CSD improves the throughput from 501 Kops/s to 721 Kops/s when the host and CSD have the same computing capability (x equals one). The improvement is attributed to two factors. First, in-CSD compaction reduces the I/O overhead by avoiding data transfer between the host and the SSD. Second, vanilla LevelDB executes buffer flush and compaction tasks on the same thread, so compaction offloading can accelerate flush operations. Even when the CSD's computing power is one-third of the host CPU's, LevelDB-CSD achieves comparable throughput to vanilla LevelDB. In addition, Figure 14 reveals that LevelDB-CSD reduces host CPU usage and stabilizes CPU consumption.

Impact of system-level page cache. Figure 15 exhibits the I/O bandwidth during the write-intensive load phase of LevelDB-CSD and vanilla LevelDB. In LevelDB-CSD, external I/Os (between the host and the SSD) are minimal, but CSD-internal I/Os are heavier than the external I/Os in vanilla LevelDB. This is because vanilla LevelDB can utilize the host page cache to reduce I/Os during compaction, while LevelDB-CSD performs compaction via CSD-internal I/Os.

Figure 13 indicates that when the workload involves reads and writes, the throughput of LevelDB-CSD lags behind that of vanilla LevelDB. These results are consistent with Figure 16, which shows LevelDB-CSD has worse read latency than vanilla LevelDB. Given that the working sets in the tests are relatively small (~300MB), they can be entirely cached in the page cache in vanilla LevelDB. However, in LevelDB-CSD, newly created SSTables by compaction operations reside in the CSD and the following user reads on them causes I/O overhead. These results reveal that CSD offloading may compromise system performance due to under-utilization of the host-side page cache, despite the benefits in reducing CPU and I/O overheads.

I/O interference in CSDs. A counterintuitive trend is observed in Figures 13 and 16: the throughput and latency of

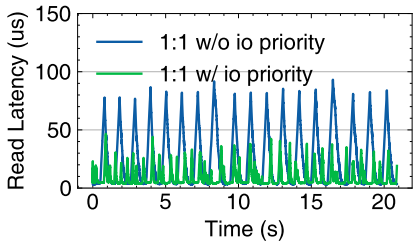


Figure 17. Impact of external I/O priority on read latency.

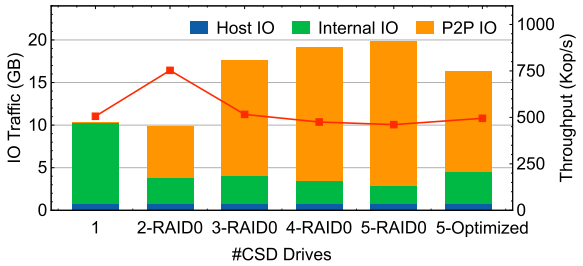


Figure 18. Throughput and I/O traffic in multi-CSD setups.

LevelDB-CSD degrade under mixed read-write workloads as the CSD's computing power increases. In particular, when x equals one, the throughput of LevelDB-CSD is only half that of vanilla LevelDB under a 50% write workload. This degradation results from I/O interference in the CSD. In-CSD compaction is an I/O-intensive task and generates I/O inside the CSD, which have a shorter processing path than external I/Os submitted by the host. The internal I/Os compete heavily for flash storage bandwidth, slowing down host I/Os. To verify the analysis, we optimize the CSD FTL to allocate a dedicated queue for internal I/Os and prioritize external I/Os. As shown in Figures 13 and 17, the optimization significantly improves system-level throughput and I/O latency of LevelDB-CSD (x equals one) under mixed workloads by addressing I/O interference.

Inter-CSD P2P transfer. We also evaluate LevelDB-CSD in a multi-CSD setup with two deployment approaches. The first runs separate LevelDB instances per CSD, while the second is to distribute a single instance across multiple CSDs organized in RAID-0. With the first approach, Figure 10 (§5.4) shows that the system throughput scales almost linearly with the number of CSDs. With the second approach, each SSTable is spread over multiple CSDs according to the RAID-0 layout. During the compaction of SSTables, PCIe P2P feature is used to transfer data between CSDs bypassing the host. Each P2P transfer consumes the interface bandwidth of both the source and destination CSDs.

Figure 18 exhibits the I/O traffic breakdown and system throughput under different numbers of CSDs in a 100% write workload. There are three kinds of I/Os: host I/Os, CSD-internal I/Os, and cross-CSD P2P I/Os. The host I/O traffic stems from buffer flush and remains constant. The P2P I/O traffic increases with the number of CSDs. Due to this overhead, system throughput decreases as the number of CSDs grows beyond 2.

To reduce P2P I/O overhead, we optimize the data layout across CSDs with a simple method. A compaction operation merges the contents of the picked SSTables with overlapping key ranges to produce a sequence of new SSTables in a batch. We leverage this locality to write 16 consecutively created SSTables to the same CSD, while SSTables from different batches are cyclically distributed across the CSDs. When a compaction operation is triggered, it is offloaded into the CSD with the highest number of input SSTables. Figure 18 (labeled "5-Optimized") demonstrates that this optimization reduces P2P I/O traffic from 17.3GB to 12GB under a five-CSD setup and improves system throughput by 7.4%.

From the LevelDB case study, two conclusions and insights can be drawn. *First, CEMU can flexibly emulate CSDs with different computing capability configurations to examine the benefits of in-storage processing and validate CSD research ideas. Second, while CSD offloading has several advantages in reducing CPU and I/O overhead, it presents some side effects, such as sacrificing the host caching benefit, introducing I/O interference in CSDs, and inter-CSD data transfer overhead. Hence, we must use CSDs in a judicious and cautious manner with countermeasures to alleviate adverse effects.*

7 Conclusion

This paper introduces CEMU, a novel CSD emulation platform that allows full-system research while addressing the fragmentation of research platforms. The CSD emulator has high configurability to model different storage and compute environments and construct multi-CSD systems without hardware dependencies. A CSD-oriented software stack is developed to manage and access the CSD in a user-friendly manner, and it is compatible with recent industry standards and the current I/O stack. We will release CEMU as an open-source tool, expecting it to enable researchers to explore new ideas on CSD systems more easily.

Acknowledgments

We thank the anonymous reviewers and the shepherd, Dr. Junyi Shu, for their insightful comments. We are also grateful to Prof. Qiang Cao (HUST) for his valuable comments on the draft, to Jin Zhao (HUST) for sharing the hardware devices, and to Sicen Li (ByteDance) for his early engineering contributions. This work was supported in part by the National Key R&D Program of China (No. 2023YFB4502901), the Shenzhen Key R&D Program (No. KJZD20240903102459001), the National Natural Science Foundation of China (No. 62302465, 62502170, 62372197, and UA22A2071), the Shandong Provincial Natural Science Foundation (No.ZR2024LZH004), the Shenzhen Natural Science Foundation (No. JCYJ20250604190909013), the China Postdoctoral Science Foundation (No. 2024M751011), and the Postdoctoral Project of Hubei Province (No. 2024HBBHCXA027).

References

[1] 2014. LevelDB. https://github.com/google/leveldb.

[2] 2017. Github - FEMU. https://github.com/MoatLab/FEMU.

[3] 2020. *ScaleFlux CSD2000*. https://scaleflux.com/products/csd-2000.

[4] 2024. NVM Express Announces the Release of the Computational Storage Feature. https://nvmexpress.org/nvm-express-announces-the-release-of-the-computational-storage-feature/.

[5] 2025. dlopen(3) — Linux manual page. https://man7.org/linux/man-pages/man3/dlopen.3.html.

[6] 2025. QEMU. https://www.qemu.org/.

[7] 2025. ubpf - Userspace eBPF VM. https://github.com/iovisor/ubpf.

[8] 2025. What Is eBPF? An Introduction and Deep Dive into the eBPF Technology. https://ebpf.io/what-is-ebpf/.

[9] 2025. What Is Io_uring? — Lord of the Io_uring Documentation. https://unixism.net/loti/what_is_io_uring.html.

[10] 2025. Xilinx/Vitis_Libraries: Vitis Libraries. https://github.com/Xilinx/Vitis_Libraries.

[11] Sandeep R. Agrawal, Sam Idicula, Arun Raghavan, Evangelos Vlachos, Venkatraman Govindaraju, Venkatanathan Varadarajan, Cagri Balkesen, Georgios Giannikis, Charlie Roth, Nipun Agarwal, and Eric Sedlar. 2017. A many-core architecture for in-memory data processing. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017, Cambridge, MA, USA, October 14-18, 2017*, Hillery C. Hunter, Jaime Moreno, Joel S. Emer, and Daniel Sánchez (Eds.). ACM, 245–258. doi:10.1145/3123939.3123985

[12] Antonio Barbalace, Martin Decky, Javier Picorel, and Pramod Bhatotia. 2020. blockNDP: Block-storage Near Data Processing. In *Middleware '20: Proceedings of the 21st International Middleware Conference Industrial Track, Delft, The Netherlands, December 7-11, 2020*. ACM, 8–15. doi:10.1145/3429357.3430519

[13] Simona Boboila, Youngjae Kim, Sudharshan S. Vazhkudai, Peter Desnoyers, and Galen M. Shipman. 2012. Active Flash: Out-of-core data analytics on flash storage. In *IEEE 28th Symposium on Mass Storage Systems and Technologies, MSST 2012, April 16-20, 2012, Asilomar Conference Grounds, Pacific Grove, CA, USA*. IEEE Computer Society, 1–12. doi:10.1109/MSST.2012.6232366

[14] Amirali Boroumand, Saugata Ghose, Youngsok Kim, Rachata Ausavarungnirun, Eric Shiu, Rahul Thakur, Daehyun Kim, Aki Kuusela, Allan Knies, Parthasarathy Ranganathan, and Onur Mutlu. 2018. Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24-28, 2018*, Xipeng Shen, James Tuck, Ricardo Bianchini, and Vivek Sarkar (Eds.). ACM, 316–331. doi:10.1145/3173162.3173177

[15] Wei Cao, Yang Liu, Zhushi Cheng, Ning Zheng, Wei Li, Wenjie Wu, Linqiang Ouyang, Peng Wang, Yijing Wang, Ray Kuan, Zhenjun Liu, Feng Zhu, and Tong Zhang. 2020. POLARDB Meets Computational Storage: Efficiently Support Analytical Workloads in Cloud-Native Relational Database. In *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, Sam H. Noh and Brent Welch (Eds.). USENIX Association, 29–41. https://www.usenix.org/conference/fast20/presentation/cao-wei

[16] Kangqi Chen, Andreas Kosmas Kakolyris, Rakesh Nadig, Manos Frouzakis, Nika Mansouri-Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, and Onur Mutlu. 2025. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. *CoRR* abs/2506.16444 (2025). arXiv:2506.16444 doi:10.48550/ARXIV.2506.16444

[17] Xiang Chen, Tao Lu, Jiapin Wang, Yu Zhong, Guangchun Xie, Xueming Cao, Yuanpeng Ma, Bing Si, Feng Ding, Ying Yang, Yunxin Huang, Yafei Yang, You Zhou, and Fei Wu. 2024. HA-CSD: Host and SSD Coordinated Compression for Capacity and Performance. In *IEEE International Parallel and Distributed Processing Symposium, IPDPS 2024, San Francisco, CA, USA, May 27-31, 2024*. IEEE, 825–838. doi:10.1109/IPDPS57955.2024.00078

[18] Yann Collet and Murray S. Kucherawy. 2021. Zstandard Compression and the 'application/zstd' Media Type. *RFC* 8878 (2021), 1–45. doi:10.17487/RFC8878

[19] Jaeyoung Do, Yang-Suk Kee, Jignesh M. Patel, Chanik Park, Kwanghyun Park, and David J. DeWitt. 2013. Query processing on smart SSDs: opportunities and challenges. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). ACM, 1221–1230. doi:10.1145/2463676.2465295

[20] Zhuohui Duan, Hao Feng, Haikun Liu, Xiaofei Liao, Hai Jin, and Bangyu Li. 2025. AegonKV: A High Bandwidth, Low Tail Latency, and Low Storage Cost KV-Separated LSM Store with SmartSSD-based GC Offloading. In *23rd USENIX Conference on File and Storage Technologies, FAST 2025, Santa Clara, CA, February 25-27, 2025*, Haryadi S. Gunawi and Vasily Tarasov (Eds.). USENIX Association, 321–335. https://www.usenix.org/conference/fast25/presentation/duan

[21] Anna Giannakou, Damian Hazen, Bjoern Enders, Lavanya Ramakrishnan, and Nicholas J. Wright. 2024. Understanding Data Movement Patterns in HPC: A NERSC Case Study. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC 2024, Atlanta, GA, USA, November 17-22, 2024*. IEEE, 71. doi:10.1109/SC41406.2024.00077

[22] Donghyun Gouk, Miryeong Kwon, Jie Zhang, Sungjoon Koh, Wonil Choi, Nam Sung Kim, Mahmut T. Kandemir, and Myoungsoo Jung. 2018. Amber*: Enabling Precise Full-System Simulation with Detailed Modeling of All SSD Resources. In *51st Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2018, Fukuoka, Japan, October 20-24, 2018*. IEEE Computer Society, 469–481. doi:10.1109/MICRO.2018.00045

[23] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. 2021. Data Movement Is All You Need: A Case Study on Optimizing Transformers. In *Proceedings of the Fourth Conference on Machine Learning and Systems, MLSys 2021, virtual, April 5-9, 2021*, Alex Smola, Alex Dimakis, and Ion Stoica (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2021/hash/bc86e95606a6392f51f95a8de106728d-Abstract.html

[24] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. 2024. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024, Edinburgh, United Kingdom, March 2-6, 2024*. IEEE, 345–360. doi:10.1109/HPCA57654.2024.00034

[25] Insoon Jo, Duck-Ho Bae, Andre S. Yoon, Jeong-Uk Kang, Sangyeun Cho, Daniel D. G. Lee, and Jaeheon Jeong. 2016. YourSQL: A High-Performance Database System Leveraging In-Storage Computing. *Proc. VLDB Endow.* 9, 12 (2016), 924–935. doi:10.14778/2994509.2994512

[26] Sang Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, Shuotao Xu, and Arvind. 2015. BlueDBM: an appliance for big data analytics. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture, Portland, OR, USA, June 13-17, 2015*, Deborah T. Marr and David H. Albonesi (Eds.). ACM, 1–13. doi:10.1145/2749469.2750412

[27] Sang Woo Jun, Andy Wright, Sizhuo Zhang, Shuotao Xu, and Arvind. 2018. GraFBoost: Using Accelerated Flash Storage for External Graph Analytics. In *45th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*, Murali Annavaram, Timothy Mark Pinkston, and Babak Falsafi (Eds.). IEEE Computer Society, 411–424. doi:10.1109/ISCA.2018.00042

[28] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seong-Yeob Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. NVMeVirt: A Versatile Software-defined Virtual NVMe Device. In *21st USENIX Conference on File and Storage Technologies, FAST 2023, Santa Clara, CA, USA, February 21-23,*

2023, Ashvin Goel and Dalit Naor (Eds.). USENIX Association, 379–394. https://www.usenix.org/conference/fast23/presentation/kim-sang-hoon

[29] Kim Malone. 2022. NVMe Computational Storage - An Update on the Standard | SNIA. https://www.snia.org/educational-library/nvme-computational-storage-update-standard-2022.

[30] Gunjae Koo, Kiran Kumar Matam, Te I, H. V. Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annavaram. 2017. Summarizer: trading communication with computing near storage. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2017, Cambridge, MA, USA, October 14-18, 2017*, Hillery C. Hunter, Jaime Moreno, Joel S. Emer, and Daniel Sánchez (Eds.). ACM, 219–231. doi:10.1145/3123939.3124553

[31] Jaewook Kwak, Sangjin Lee, Kibin Park, Jinwoo Jeong, and Yong Ho Song. 2020. Cosmos+ OpenSSD: Rapid Prototype for Flash Storage Systems. *ACM Trans. Storage* 16, 3 (2020), 15:1–15:35. doi:10.1145/3385073

[32] Kitaek Lee, Insoon Jo, Jaechan Ahn, Hyuk Lee, Hwang Lee, Woong Sul, and Hyungsoo Jung. 2023. Deploying Computational Storage for HTAP DBMSs Takes More Than Just Computation Offloading. *Proc. VLDB Endow.* 16, 6 (2023), 1480–1493. doi:10.14778/3583140.3583161

[33] Yunjae Lee, Hyeseong Kim, and Minsoo Rhu. 2024. PreSto: An In-Storage Data Preprocessing System for Training Recommendation Models. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 340–353. doi:10.1109/ISCA59077.2024.00033

[34] Cangyuan Li, Ying Wang, Cheng Liu, Shengwen Liang, Huawei Li, and Xiaowei Li. 2021. GLIST: Towards In-Storage Graph Learning. In *Proceedings of the 2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021*, Irina Calciu and Geoff Kuenning (Eds.). USENIX Association, 225–238. https://www.usenix.org/conference/atc21/presentation/li-cangyuan

[35] Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Matias Bjørling, and Haryadi S. Gunawi. 2018. The CASE of FEMU: Cheap, Accurate, Scalable and Extensible Flash Emulator. In *16th USENIX Conference on File and Storage Technologies, FAST 2018, Oakland, CA, USA, February 12-15, 2018*, Nitin Agrawal and Raju Rangaswami (Eds.). USENIX Association, 83–90. https://www.usenix.org/conference/fast18/presentation/li

[36] Siqi Li, Fengbin Tu, Liu Liu, Jilan Lin, Zheng Wang, Yangwook Kang, Yufei Ding, and Yuan Xie. 2023. ECSSD: Hardware/Data Layout Co-Designed In-Storage-Computing Architecture for Extreme Classification. In *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, Yan Solihin and Mark A. Heinrich (Eds.). ACM, 58:1–58:14. doi:10.1145/3579371.3589093

[37] Shiyu Li, Yitu Wang, Edward Hanson, Andrew Chang, Yang-Seok Ki, Hai Li, and Yiran Chen. 2024. NDRec: A Near-Data Processing System for Training Large-Scale Recommendation Models. *IEEE Trans. Computers* 73, 5 (2024), 1248–1261. doi:10.1109/TC.2024.3365939

[38] Shengwen Liang, Ying Wang, Youyou Lu, Zhe Yang, Huawei Li, and Xiaowei Li. 2019. Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval. In *Proceedings of the 2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, Dahlia Malkhi and Dan Tsafrir (Eds.). USENIX Association, 395–410. https://www.usenix.org/conference/atc19/presentation/liang

[39] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jerónimo Castrillón, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Marjan Fariborz, Amin Farmahini Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc S. Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. 2020. The gem5 Simulator: Version 20.0+. *CoRR* abs/2007.03152 (2020). arXiv:2007.03152 https://arxiv.org/abs/2007.03152

[40] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyan Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-Mei Hwu. 2019. DeepStore: In-Storage Acceleration for Intelligent Queries. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*. ACM, 224–238. doi:10.1145/3352460.3358320

[41] Nika Mansouri-Ghiasi, Jisung Park, Harun Mustafa, Jeremie S. Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. 2022. GenStore: a high-performance in-storage processing system for genome sequence analysis. In *ASPLOS '22: 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, 28 February 2022 - 4 March 2022*, Babak Falsafi, Michael Ferdman, Shan Lu, and Thomas F. Wenisch (Eds.). ACM, 635–654. doi:10.1145/3503222.3507702

[42] Nika Mansouri-Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Mao, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. 2024. MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 660–677. doi:10.1109/ISCA59077.2024.00054

[43] Bill Martin and Stephen Bates. 2022. Computational Storage Architecture and Programming Model | SNIA | Experts on Data. https://www.snia.org/csarch https://www.snia.org/csarch.

[44] Hasan Al Maruf and Mosharaf Chowdhury. 2023. Memory Disaggregation: Advances and Open Challenges. *CoRR* abs/2305.03943 (2023). arXiv:2305.03943 doi:10.48550/ARXIV.2305.03943

[45] Kiran Kumar Matam, Gunjae Koo, Haipeng Zha, Hung-Wei Tseng, and Murali Annavaram. 2019. GraphSSD: graph semantics aware SSD. In *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*, Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman (Eds.). ACM, 116–128. doi:10.1145/3307650.3322275

[46] Onur Mutlu, Ataberk Olgun, and Ismail Emir Yüksel. 2025. Memory-Centric Computing: Solving Computing's Memory Problem. In *IEEE International Memory Workshop, IMW 2025, Monterey, CA, USA, May 18-21, 2025*. IEEE, 1–4. doi:10.1109/IMW61990.2025.11026935

[47] Fuping Niu, Jianhui Yue, Jiangqiu Shen, Xiaofei Liao, and Hai Jin. 2024. FlashGNN: An In-SSD Accelerator for GNN Training. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024, Edinburgh, United Kingdom, March 2-6, 2024*. IEEE, 361–378. doi:10.1109/HPCA57654.2024.00035

[48] Geraldo F. Oliveira, Ataberk Olgun, Abdullah Giray Yaglikçi, F. Nisa Bostanci, Juan Gómez-Luna, Saugata Ghose, and Onur Mutlu. 2024. MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024,*

Edinburgh, United Kingdom, March 2-6, 2024. IEEE, 186–203. doi:10.1109/HPCA57654.2024.00024

[49] Yifan Qiao, Xubin Chen, Ning Zheng, Jiangpeng Li, Yang Liu, and Tong Zhang. 2022. Closing the B+-tree vs. LSM-tree Write Amplification Gap on Modern Storage Hardware with Built-in Transparent Compression. In *20th USENIX Conference on File and Storage Technologies, FAST 2022, Santa Clara, CA, USA, February 22-24, 2022*, Dean Hildebrand and Donald E. Porter (Eds.). USENIX Association, 69–82. https://www.usenix.org/conference/fast22/presentation/qiao

[50] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*, Bronis R. de Supinski, Mary W. Hall, and Todd Gamblin (Eds.). ACM, 59. doi:10.1145/3458817.3476205

[51] Zhenyuan Ruan, Tong He, and Jason Cong. 2019. INSIDER: Designing In-Storage Computing System for Emerging High-Performance Drive. In *Proceedings of the 2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, Dahlia Malkhi and Dan Tsafrir (Eds.). USENIX Association, 379–394. https://www.usenix.org/conference/atc19/presentation/ruan

[52] Samsung. 2022. Samsung Electronics Develops Second-Generation SmartSSD Computational Storage Drive with Upgraded Processing Functionality. https://semiconductor.samsung.com/news-events/news/samsung-electronics-develops-second-generation-smartssd-computational-storage-drive-with-upgraded-processing-functionality/

[53] Subhadeep Sarkar, Kaijie Chen, Zichen Zhu, and Manos Athanassoulis. 2022. Compactionary: A Dictionary for LSM Compactions. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 2429–2432. doi:10.1145/3514221.3520169

[54] ScaleFlux. 2025. ScaleFlux Enterprise NVMe SSDs with Write Reduction Technology. https://scaleflux.com/products/.

[55] SNIA. 2013. Computational Storage API v1.0. https://www.snia.org/csapi.

[56] Jingbo Su, Jiahao Li, Luofan Chen, Cheng Li, Kai Zhang, Liang Yang, Sam H. Noh, and Yinlong Xu. 2023. Revitalizing the Forgotten On-Chip DMA to Expedite Data Movement in NVM-based Storage Systems. In *21st USENIX Conference on File and Storage Technologies, FAST 2023, Santa Clara, CA, USA, February 21-23, 2023*, Ashvin Goel and Dalit Naor (Eds.). USENIX Association, 363–378. https://www.usenix.org/conference/fast23/presentation/su

[57] Hui Sun, Wei Liu, Zhi Qiao, Song Fu, and Weisong Shi. 2018. DStore: A Holistic Key-Value Store Exploring Near-Data Processing and On-Demand Scheduling for Compaction Optimization. *IEEE Access* 6 (2018), 61233–61253. doi:10.1109/ACCESS.2018.2873579

[58] Xuan Sun, Hu Wan, Qiao Li, Chia-Lin Yang, Tei-Wei Kuo, and Chun Jason Xue. 2022. RM-SSD: In-Storage Computing for Large-Scale Recommendation Inference. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*. IEEE, 1056–1070. doi:10.1109/HPCA53966.2022.00081

[59] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. 2018. MQSim: A Framework for Enabling Realistic Studies of Modern Multi-Queue SSD Devices. In *16th USENIX Conference on File and Storage Technologies, FAST 2018, Oakland, CA, USA, February 12-15, 2018*, Nitin Agrawal and Raju Rangaswami (Eds.). USENIX Association, 49–66. https://www.usenix.org/conference/fast18/presentation/tavakkol

[60] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024*, Saurabh Bagchi and Yiying Zhang (Eds.). USENIX Association, 1135–1150. https://www.usenix.org/conference/atc24/presentation/tian

[61] Devesh Tiwari, Simona Boboila, Sudharshan S. Vazhkudai, Youngjae Kim, Xiaosong Ma, Peter Desnoyers, and Yan Solihin. 2013. Active flash: towards energy-efficient, in-situ data analytics on extreme-scale machines. In *Proceedings of the 11th USENIX conference on File and Storage Technologies, FAST 2013, San Jose, CA, USA, February 12-15, 2013*, Keith A. Smith and Yuanyuan Zhou (Eds.). USENIX, 119–132. https://www.usenix.org/conference/fast13/technical-sessions/presentation/tiwari

[62] Devesh Tiwari, Sudharshan S. Vazhkudai, Youngjae Kim, Xiaosong Ma, Simona Boboila, and Peter Desnoyers. 2012. Reducing Data Movement Costs Using Energy-Efficient, Active Computation on SSD. In *2012 Workshop on Power-Aware Computing Systems, HotPower'12, Hollywood, CA, USA, October 7, 2012*, Mani Srivastava and Thomas F. Wenisch (Eds.). USENIX Association. https://www.usenix.org/conference/hotpower12/workshop-program/presentation/tiwari

[63] Mahdi Torabzadehkashi, Ali Heydarigorji, Siavash Rezaei, Hossein Bobarshad, Vladimir Castro Alves, and Nader Bagherzadeh. 2019. Accelerating HPC Applications Using Computational Storage Devices. In *21st IEEE International Conference on High Performance Computing and Communications; 17th IEEE International Conference on Smart City; 5th IEEE International Conference on Data Science and Systems, HPCC/SmartCity/DSS 2019, Zhangjiajie, China, August 10-12, 2019*, Zheng Xiao, Laurence T. Yang, Pavan Balaji, Tao Li, Keqin Li, and Albert Y. Zomaya (Eds.). IEEE, 1878–1885. doi:10.1109/HPCC/SMARTCITY/DSS.2019.00259

[64] John Paul Walters, Andrew J. Younge, Dong-In Kang, Ke Thia Yao, Mikyung Kang, Stephen P. Crago, and Geoffrey C. Fox. 2014. GPU Passthrough Performance: A Comparison of KVM, Xen, VMWare ESXi, and LXC for CUDA and OpenCL Applications. In *2014 IEEE 7th International Conference on Cloud Computing, Anchorage, AK, USA, June 27 - July 2, 2014*. IEEE Computer Society, 636–643. doi:10.1109/CLOUD.2014.90

[65] Shibo Wang and Engin Ipek. 2016. Reducing data movement energy via online data clustering and encoding. In *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 32:1–32:13. doi:10.1109/MICRO.2016.7783735

[66] Yitu Wang, Shiyu Li, Qilin Zheng, Linghao Song, Zongwang Li, Andrew Chang, Hai Li, and Yiran Chen. 2024. NDSEARCH: Accelerating Graph-Traversal-Based Approximate Nearest Neighbor Search through Near Data Processing. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 368–381. doi:10.1109/ISCA59077.2024.00035

[67] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. 2024. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2024, Edinburgh, United Kingdom, March 2-6, 2024*. IEEE, 330–344. doi:10.1109/HPCA57654.2024.00033

[68] Yu Wang, Zibin Sun, You Zhou, Tao Lu, Changsheng Xie, and Fei Wu. 2024. Balloon-ZNS: Constructing High-Capacity and Low-Cost ZNS SSDs with Built-in Compression. In *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC 2024, San Francisco, CA, USA, June 23-27, 2024*, Vivek De (Ed.). ACM, 125:1–125:6. doi:10.1145/3649329.3657368

[69] Peter Wilcox and Heiner Litz. 2021. Design for computational storage simulation platform. In *CHEOPS '21: Proceedings of the Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems, In Conjunction with EuroSys 2021, Online Event, United Kingdom, April, 2021*, Michael Kuhn, Kira Duwe, Jean-Thomas Acquaviva, Konstantinos Chasapis, and Jalil Boukhobza (Eds.). ACM, 5:1–5:8. doi:10.1145/3439839.3459085

[70] Zhe Yang, Youyou Lu, Xiaojian Liao, Youmin Chen, Junru Li, Siyu He, and Jiwu Shu. 2023. λ-IO: A Unified IO Stack for Computational Storage. In *21st USENIX Conference on File and Storage Technologies, FAST 2023, Santa Clara, CA, USA, February 21-23, 2023*, Ashvin Goel and Dalit Naor (Eds.). USENIX Association, 347–362. https://www.usenix.org/conference/fast23/presentation/yang-zhe

[71] Xin You, Zhibo Xuan, Hailong Yang, Zhongzhi Luan, Yi Liu, and Depei Qian. 2024. GVARP: Detecting Performance Variance on Large-Scale Heterogeneous Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC 2024, Atlanta, GA, USA, November 17-22, 2024*. IEEE, 57. doi:10.1109/SC41406.2024.00063

[72] Jinghuan Yu, Sam H. Noh, Young-ri Choi, and Chun Jason Xue. 2023. ADOC: Automatically Harmonizing Dataflow Between Components in Log-Structured Key-Value Stores for Improved Performance. In *21st USENIX Conference on File and Storage Technologies, FAST 2023, Santa Clara, CA, USA, February 21-23, 2023*, Ashvin Goel and Dalit Naor (Eds.). USENIX Association, 65–80. https://www.usenix.org/conference/fast23/presentation/yu

[73] Jian Zhang, Yujie Ren, Marie Nguyen, Changwoo Min, and Sudarsun Kannan. 2024. OmniCache: Collaborative Caching for Near-storage Accelerators. In *22nd USENIX Conference on File and Storage Technologies, FAST 2024, Santa Clara, CA, USA, February 27-29, 2024*, Xiaosong Ma and Youjip Won (Eds.). USENIX Association, 35–50. https://www.usenix.org/conference/fast24/presentation/zhang-jian

[74] Chen Zou and Andrew A. Chien. 2022. ASSASIN: Architecture Support for Stream Computing to Accelerate Computational Storage. In *55th IEEE/ACM International Symposium on Microarchitecture, MICRO 2022, Chicago, IL, USA, October 1-5, 2022*. IEEE, 354–368. doi:10.1109/MICRO56248.2022.00035

A Artifact Appendix

A.1 Abstract

The artifact comprises the CEMU source code, test programs, and documentation. Developed as an extension of QEMU, CEMU leverages host CPU and DRAM resources to emulate a CSD for guest virtual machines. While CEMU runs on standard x86 servers, for multi-CSD emulation we recommend a system equipped with at least 40 CPU cores and 512 GB of DRAM. To simplify the evaluation process, we provide a preconfigured guest VM image along with automated test scripts.

A.2 Artifact check-list (meta-information)

- **Run-time environment:** Ubuntu 22.04.
- **Hardware:** >=40 x86 CPU cores and >=512GB DRAM.
- **Experiments:** Figure 8 and Figure 10.
- **How much disk space required (approximately)?:** 30GB.
- **How much time is needed to prepare workflow (approximately)?:** 20 minutes.
- **How much time is needed to complete experiments (approximately)?:** 20 minutes.
- **Publicly available?:** Yes.

A.3 Description

A.3.1 How to access. The CEMU codebase is organized into three repositories, all of which are open-sourced on GitHub:

- **CEMU:** https://github.com/cs-qyzhang/CEMU. A CSD emulator built upon QEMU and FEMU. This is the primary repository and contains the project documentation.
- **linux-cemu:** https://github.com/cs-qyzhang/linux-cemu. An implementation of the CEMU software stack based on the Linux 6.8 kernel. This repository includes NVMe CSD driver adaptations, the FDMFS implementation, and io_uring integration.
- **ubpf-cemu:** https://github.com/cs-qyzhang/ubpf-cemu. The eBPF execution environment, built on uBPF.

A.4 Installation

First, clone the three repositories and install the required dependencies.

```
~$ mkdir cemu && cd cemu
~/cemu$ git clone https://github.com/cs-qyzhang/CEMU.git
~/cemu$ git clone
    ↪ https://github.com/cs-qyzhang/linux-cemu.git
~/cemu$ git clone
    ↪ https://github.com/cs-qyzhang/ubpf-cemu.git
    ↪ --recurse-submodules
~/cemu$ sudo ./CEMU/femu-scripts/pkgdep.sh
```

Next, build CEMU. Note that you must compile ubpf-cemu before building CEMU.

```
# compile modified linux kernel
~/cemu$ cd linux-cemu
~/cemu/linux-cemu$ make bzImage -j$(nproc)
~/cemu/linux-cemu$ cd ..

# compile ubpf for executing eBPF programs
~/cemu$ cd ubpf-cemu
~/cemu/ubpf-cemu$ cmake -S . -B build
    ↪ -DUBPF_ENABLE_TESTS=true
~/cemu/ubpf-cemu$ cmake --build build --config Release
~/cemu/ubpf-cemu$ cd ..

# compile CEMU emulator
~/cemu$ cd CEMU
~/cemu/CEMU$ mkdir build && cd build
~/cemu/CEMU/build$ ../femu-scripts/femu-copy-scripts.sh .
~/cemu/CEMU/build$ ./femu-compile.sh

# compile shared-library type CSF
~/cemu/CEMU/build$ cd ../tests/cemu
~/cemu/CEMU/tests/cemu$ make kernel
```

Finally, download the prebuilt VM image.

```
~/cemu/CEMU/tests/cemu$ cd ../../../
~/cemu$ wget https://bit.ly/cemu-dqib -O dqib.tar.xz
~/cemu$ tar -xJf dqib.tar.xz
```

A.5 Experiment workflow

Start CEMU virtual machine.

```
~/cemu$ cd CEMU/build
~/cemu/CEMU/build$ ./run-csd.sh
```

After VM is up, open another terminal and connect to CEMU via ssh. The username and password of prebuilt VM images are `root`.

```
# open another terminal
~$ ssh root@localhost -p 2222
```

In the guest VM, mount the CSD drive. Note that the `cemu_mount.sh` script must be run each time the VM is started.

```
# inside VM
root@debian:~# ./cemu-mount.sh
```

Now you will see three NVMe namespaces exposed by the NVMe CSD and two file systems corresponding to NAND and device memory:

- **`/dev/nvme0n1`**: NVM namespace, representing the NAND flash inside the CSD. This is equivalent to a traditional NVMe SSD.
- **`/dev/nvme0m2`**: Memory namespace, representing the device memory inside the CSD. By abstracting device memory as a block device and mounting FDMFS on top of it, users can access device memory via file-based operations such as `pread/pwrite`, and enable in-device data copies between NAND and on-device memory via the `copy_file_range` system call.
- **`/dev/nvme0c3`**: Compute namespace. CEMU uses this namespace to manage CSFs.

```
# inside VM
root@debian:~# lsblk
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0    0   20G  0 disk
`-sda1    8:1    0   20G  0 part /
sr0      11:0    1 1024M  0 rom
nvme0m2 252:0    0    2G  0 disk /mnt/fdm0
nvme0n1 259:0    0   48G  0 disk /mnt/nvme0

root@debian:~# mount | grep 'nvme\|cemu'
cemusrc on /root/CEMU type 9p
     ↪ (rw,relatime,access=client,trans=virtio)
/dev/nvme0n1 on /mnt/nvme0 type ext4 (rw,relatime)
/dev/nvme0m2 on /mnt/fdm0 type fdmfs (rw,relatime)
```

The test and benchmark programs are located in `CEMU/tests/cemu`. You need to build them once before running them for the first time:

```
# inside VM
root@debian:~# cd CEMU/tests/cemu
root@debian:~CEMU/tests/cemu# make
```

We provide several examples to demonstrate how to use the CEMU software stack.

```
# inside VM
root@debian:~CEMU/tests/cemu# ./build/test_fdmfs
root@debian:~CEMU/tests/cemu# ./build/vadd_example
     ↪ --src-in-file --dst-in-file
root@debian:~CEMU/tests/cemu# ./build/test_indirect
root@debian:~CEMU/tests/cemu# ./build/test_p2p
```

A.6 Evaluation and expected results

A.6.1 Micro benchmark. This experiment reproduces the micro-benchmark results reported in Figure 8a & 8b of the paper. We evaluate four representative CSD applications, including `lz4`, `grep`, `knn`, and `sql`.

All experiments are conducted under the *freezing mode* of CEMU (§3.2.3) to ensure deterministic execution and stable performance measurements. Each application is executed independently, and no other workload is running concurrently inside the VM.

For each application, we vary the concurrency and measure the end-to-end execution time. Specifically, each application is executed serially with concurrent chunks of 1, 2, and 4. We first run tests on a single drive, then repeat the experiments on three drives, keeping the total workload size the same in both cases.

To start the experiment, launch the CEMU virtual machine with freezing mode enabled:

```
~/cemu$ cd CEMU/build
~/cemu/CEMU/build$ ./run-3csd-freezing.sh
```

Then, log into the guest VM and mount the CEMU devices:

```
# open another terminal
~$ ssh root@localhost -p 2222
# inside VM
root@debian:~# ./cemu-mount.sh
```

Next, execute the four micro-benchmark applications sequentially:

```
# inside VM
root@debian:~# cd CEMU/tests/cemu
root@debian:~# ./test_lz4.sh
root@debian:~# ./test_grep.sh
root@debian:~# ./test_knn.sh
root@debian:~# ./test_sql.sh
```

Each application prints a line containing "Time: " in its output, which represents the end-to-end execution time of a single run. These reported times are expected to be close to the corresponding 1C/2C/4C bars for each application in Figure 8a and Figure 8b.

A.6.2 Scalability evaluation. This experiment evaluates the scalability of CEMU with an increasing number of CSDs, corresponding to Figure 10. We focus on the `lz4` workload.

The number of emulated CSDs is varied from 1 to 8. For each configuration, the same workload is distributed across all disks, and we measure the aggregated end-to-end throughput.

To start the scalability experiment, launch the CEMU virtual machine with eight CSDs enabled:

```
~/cemu$ cd CEMU/build
~/cemu/CEMU/build$ ./run-8csd-freezing.sh
```

Then, log into the guest VM and mount the CEMU devices:

```
# open another terminal
~$ ssh root@localhost -p 2222
# inside VM
root@debian:~# ./cemu-mount.sh
```

Finally, run the multi-CSD `lz4` benchmark:

```
#inside VM
root@debian:~# cd CEMU/tests/cemu
root@debian:~# ./test_multicsd_lz4.sh
```

The script iterates over configurations with 1 to 8 drives and prints the total throughput for each case. In each run, the output line containing "BW: " reports the aggregated system throughput across all CSDs, which increases with the number of disks and matches the scalability trend shown in Figure 10.